

AI 비전검수 시스템

상태머신 · 노드 책임 · 정상 운전 · 오류 복구 워크플로우

MasterNode · ControlNode · VisionNode · LogNode / 2층·1층 컨베이어 / 양면 정지 촬영

검사 방식	상층 한 면 검사 → FLIPPING → 하층 반대 면 검사 → Sensor 3 판정 잠금 → 분류
정합성 원본	제품 ID·활성 FIFO·최종 판정은 MasterNode 단일 소유
실물 제어	Arduino Mega 2560 + TB6600. ControlNode가 실제 장비 상태 소유
비전	정지 촬영, 하드웨어 트리거. Vision은 카메라 ARM·수집, Control/Mega는 트리거·조명
오류 철학	제품 문제는 FORCED_NG 후 계속, 추적 무결성 상실만 FAULT_STOP

1. 전체 시스템 흐름



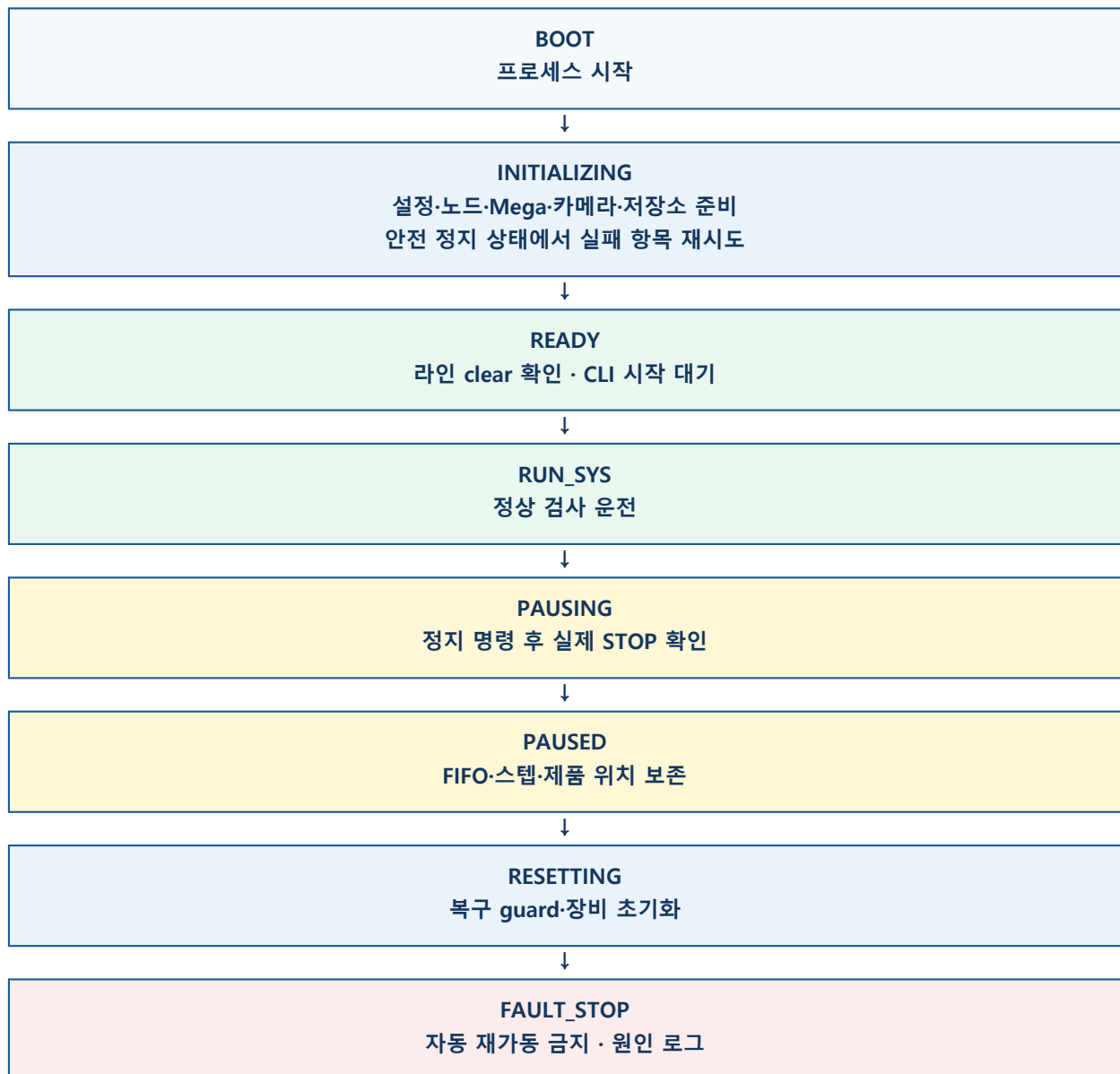
핵심: 촬영 완료 후 컨베이어는 즉시 재가동하며 추론 완료를 기다리지 않는다. Sensor 3 시점에도 결과가 없으면 라인을 멈추지 않고 해당 제품만 FORCED_NG로 고정한다.

2. 노드 책임과 상태 소유권

MasterNode	SystemState, Active FIFO, ProductContext, 최종 판정	제품 ID 발급 · 센서 이벤트와 제품 연결 · 공정 조율 · FAULT_STOP 결정
ControlNode	컨베이어·센서·액추에이터·Mega-E-stop·트리거·조명 실제 상태	PUL/DIR/ENA 제어 · 스텝 추적 · 물리 이벤트 생성 · 로컬 안전 정지
VisionNode	카메라 연결 · CaptureJob · InferenceJob · StationResult	카메라 ARM·프레임 수집 · 이미지 저장 · PatchCore 추론 · 결과 발행
LogNode	SQLite 레코드 · health_state · SpoolState	제품·장비·비전·오류 이벤트 영구 저장 · 조회 API. 운전 명령 중계 없음

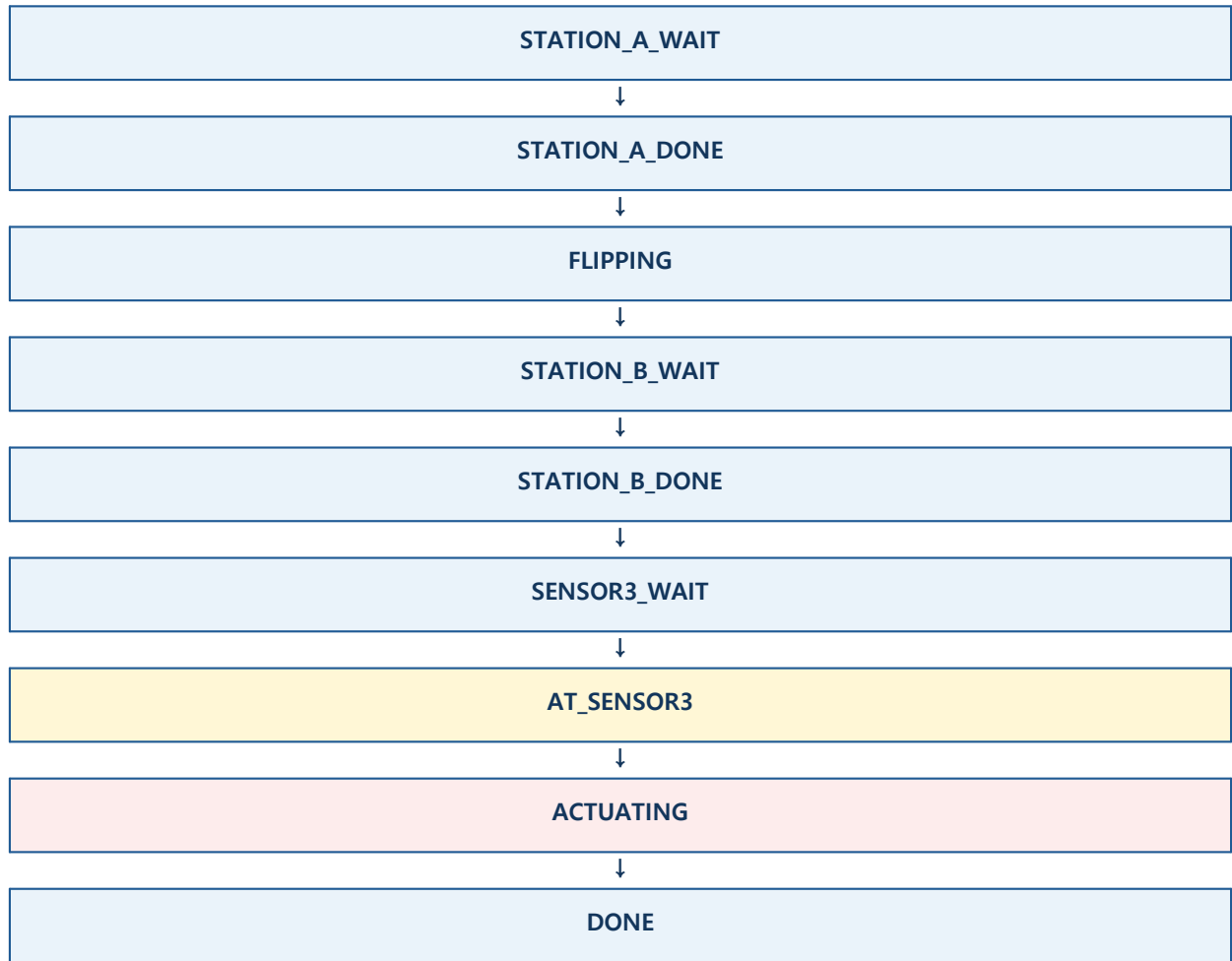
요청 상태와 실제 상태는 분리한다. Master가 RUN_CONV를 요청해도 실제 상태 확인은 ControlNode만 수행한다.
Vision-Control은 제품 FIFO를 따로 만들지 않는다.

3. 전체 시스템 FSM



정상 경로는 BOOT → INITIALIZING → READY → RUN_SYS입니다. PAUSING/PAUSED는 제품 위치를 보존하는 복구 가능 정지이고, FAULT_STOP은 RESETTING을 거쳐야 합니다. 촬영용 CAPTURE_HOLD는 시스템 PAUSED가 아니라 RUN_SYS 내부의 컨베이어 지역 상태입니다.

4. 제품 물리 FSM · 비전 비동기 관계



A/B 촬영 작업과 추론 작업은 별도 FSM입니다. 촬영이 끝난 직후 제품은 다음 물리 상태로 이동하고 추론은 QUEUED → RUNNING → DONE/FAILED로 뒤에서 계속됩니다. result_locked=true가 된 이후 도착한 결과는 LATE_IGNORED 또는 LATE_CONFLICT_IGNORED로만 기록합니다.

5. 액추에이터 · FIFO 완료 처리

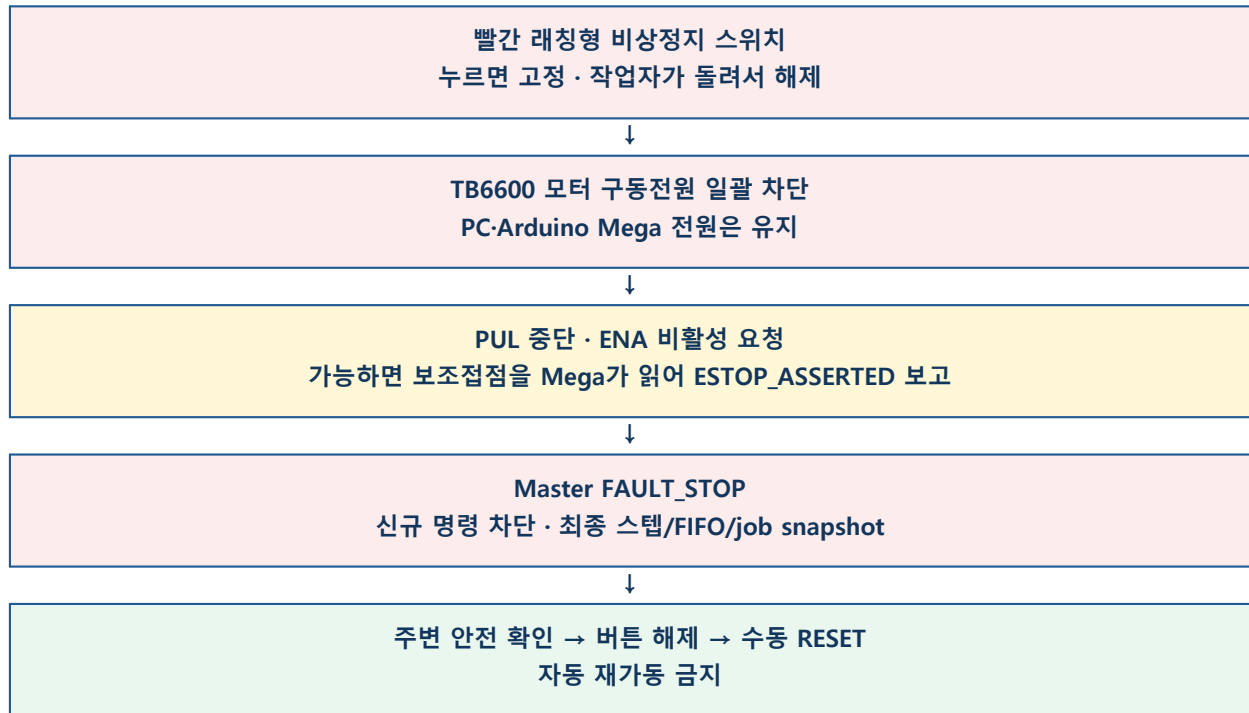


예약이 여러 개여도 물리 FIFO를 건너뛰어 제거하지 않습니다. 뒤 제품의 완료 이벤트가 먼저 처리되면 완료 표시만 보존하고, 앞 제품이 제거된 뒤 차례대로 제거합니다.

6. 오류 처리 원칙

제품 단위	촬영·추론 최종 실패, Sensor 3 결과 미완료, 동일 제품 비전결과 충돌	FORCED_NG 후 계속 운전. 반복 장치 실패만 PAUSED 복구
복구 가능한 장치	Vision 단절, 카메라 반복 실패, 위치 보존 가능한 액추에이터 오류	PAUSING → PAUSED. 소프트 복구만 자동 재개, 물리 점검은 수동 재개
추적 무결성	FIFO 불일치, 제품 ID·위치 불명, 센서 이벤트 충돌, route 불명	FAULT_STOP → LINE_CLEAR_REQUIRED → RESETING
저장 장애	SQLite 실패·spool 증가	보존 가능하면 DEGRADED 계속, 위험 한도에서 PAUSED. FAULT_STOP 없음
물리 E-stop	빨간 래칭 스위치 작동	TB6600 구동전원 물리 차단 → FAULT_STOP → 버튼 해제·수동 RESET

7. PoC 물리 비상정지



PoC 단계에서는 정식 안전 릴레이·접촉기 구성을 필수로 두지 않고, 모터 전원공급기 정격에 맞는 래칭형 차단 스위치를 적용합니다. 최종 장비 단계에서는 안전 담당자 검토와 정식 안전회로 적용 여부를 다시 판단합니다.

8. 구현 전 남은 TODO

컨베이어·센서	정지 촬영, 시간+스텝 이중 검증, 센서 stuck는 FAULT_STOP	steps/mm, 속도, 위치 offset, debounce, 정지거리
카메라·조명	Vision ARM/수집 · Control/Mega 하드웨어 트리거/조명	트리거 전압·펄스폭, 조명 채널·안정화, 3+1 배치
액추에이터	2위치 분기 암, 4개 예약, BUSY 시 PAUSED, 위치 불명 FAULT_STOP	전환시간, 목표거리, 통과확인 스텝, 정지 여유
E-stop	TB6600 구동전원 물리 차단, 해제 후 수동 RESET	전원공급기 정격, 스위치·배선, Mega 피드백 핀
실행 환경	Ubuntu 24.04 · ROS 2 Jazzy · Python 3.12	Hikrobot MVS · GPU 드라이버 · CUDA · PyTorch 버전

실행 기준: 상세 코드 작성은 데이터 구조·메시지 인터페이스·FSM 스켈레톤부터 시작하며, HARDWARE_REQUIRED 값은 hardware profile에서 누락 시 INITIALIZING을 유지합니다. 장비가 없어도 sim profile로 상태머신과 오류 주입 테스트를 진행할 수 있습니다.